Action Controller Overview

In this guide you will learn how controllers work and how they fit into the request cycle in your application.

1 What Does a Controller Do?

Action Controller is the C in MVC. After routing has determined which controller to use for a request, your controller is responsible for making sense of the request and producing the appropriate output. Luckily, Action Controller does most of the groundwork for you and uses smart conventions to make this as straightforward as possible.

For most conventional RESTful applications, the controller will receive the request (this is invisible to you as the developer), fetch or save data from a model and use a view to create HTML output. If your controller needs to do things a little differently, that's not a problem, this is just the most common way for a controller to work.
A controller can thus be thought of as a middle man between models and views. It makes the model data available to the view so it can display that data to the user, and it saves or updates data from the user to the model.

For more details on the routing process, see Rails Routing from the Outside In.

2 Controller Naming Convention

The naming convention of controllers in Rails favors pluralization of the last word in the controller's name, although it is not strictly required (e.g. `ApplicationController`). For example, `ClientsController` is preferable to `ClientController`, `SiteAdminsController` is preferable to `SiteAdminController` or `SitesAdminsController`, and so on.
Following this convention will allow you to use the default route generators (e.g. `resources`, etc) without needing to qualify each `:path` or `:controller`, and keeps URL and path helpers' usage consistent throughout your application. See Layouts & Rendering Guide for more details.
The controller naming convention differs from the naming convention of models, which are expected to be named in singular form.

3 Methods and Actions

A controller is a Ruby class which inherits from `ApplicationController` and has methods just like any other class. When your application receives a request, the routing will determine which controller and action to run, then Rails creates an instance of that controller and runs the method with the same name as the action.

```
class ClientsController < ApplicationController
  def new
  end
end
```

As an example, if a user goes to `/clients/new` in your application to add a new client, Rails will create an instance of `ClientsController` and run the `new` method. Note that the empty method from the example above would work just fine because Rails will by default render the `new.html.erb` view unless the action